ByteMart Digital

THE CODING PRACTICE SERIES

Java Coding Questions

Arrays, strings, hashing, linked lists, stacks, recursion, trees, DP & more

42 hand-picked coding interview problems, each with a worked example, a clear approach, complexity analysis, and clean, commented Java solutions.

42	42	10	Full
Problems	Solutions	Topics	Code

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Java Coding Questions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

Arrays & Two Pointers 4

Strings 11

Hashing 15

Linked Lists 18

Stacks & Queues 22

Recursion & Backtracking 25

Searching & Sorting 29

Trees 32

Bit Manipulation 37

Dynamic Programming 39

This bank contains 42 Java interview questions across 10 sections, including 42 coding challenges with full solutions. Use the bookmarks panel to jump to any section or question.

SECTION 01

Arrays & Two Pointers

Q1 Two Sum — find indices of two numbers that add to a target.

</> CODE

SOLUTION

Given an integer array and a target value, return the indices of the two numbers that add up to the target. Each input has exactly one solution, and you may not reuse the same element.

```text
Input:  nums = [2, 7, 11, 15], target = 9
Output: [0, 1]        (because nums[0] + nums[1] = 2 + 7 = 9)
```

The brute-force approach checks every pair in O(n^2). Instead, keep a hash map from value to index; for each number, look up whether its complement (target minus the number) has already been seen. That turns the search into a single pass.

```java
static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need))
            return new int[]{ seen.get(need), i };
        seen.put(nums[i], i);
    }
    return new int[0];   // no pair found
}
```

Time	Space
O(n)	O(n)

TIP

Store each number only after checking for its complement, so you never pair an element with itself.

Q2 Best Time to Buy and Sell Stock — maximize a single trade profit.

</> CODE

SOLUTION

Given daily prices, find the maximum profit from buying on one day and selling on a later day. If no profit is possible, return 0.

```text
Input:  prices = [7, 1, 5, 3, 6, 4]
Output: 5             (buy at 1, sell at 6)
```

Track the lowest price seen so far as you scan left to right. At each day, the best profit ending there is the current price minus that running minimum. Keep the maximum of those.

```java
static int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE, best = 0;
    for (int p : prices) {
        if (p < minPrice) minPrice = p;
        else best = Math.max(best, p - minPrice);
    }
    return best;
}
```

Time	Space
O(n)	O(1)

NOTE

A single pass suffices — you never need to look back, because the running minimum already captures the best buy day so far.

Q3 Move Zeroes — shift all zeros to the end in place.

`</> CODE`

SOLUTION

Move all zeros in the array to the end while keeping the relative order of the non-zero elements, modifying the array in place.

```text
Input:  nums = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

Use a write pointer. Copy each non-zero value forward to the write position, then fill the remaining slots with zeros. This preserves order in a single pass with no extra array.

```java
static void moveZeroes(int[] nums) {
    int insert = 0;
    for (int n : nums)
        if (n != 0) nums[insert++] = n;
    while (insert < nums.length)
        nums[insert++] = 0;
}
```

Time	Space
O(n)	O(1)

Q4 Container With Most Water — maximize the trapped area.

</> CODE

SOLUTION

Each array value is the height of a vertical line. Pick two lines that, together with the x-axis, hold the most water. The area is the shorter height times the distance between the lines.

```text
Input:  height = [1, 8, 6, 2, 5, 4, 8, 3, 7]
Output: 49
```

Start with the widest pair (the two ends) and move inward. At each step the area is limited by the shorter line, so moving the taller pointer can never help — always move the pointer at the shorter line, hoping for a taller one.

```java
static int maxArea(int[] height) {
    int l = 0, r = height.length - 1, max = 0;
    while (l < r) {
        int area = Math.min(height[l], height[r]) * (r - l);
        max = Math.max(max, area);
        if (height[l] < height[r]) l++;
        else r--;
    }
    return max;
}
```

Time	Space
O(n)	O(1)

KEY IDEA
The two-pointer move is justified because the width only shrinks, so the only way to beat the current area is a taller shorter-line — found by advancing the shorter side.

Q5 Maximum Subarray — Kadane's algorithm.

</> CODE

SOLUTION

Find the contiguous subarray with the largest sum and return that sum. The array can contain negative numbers.

```text
Input:  nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6             (the subarray [4, -1, 2, 1])
```

At each index decide: extend the current subarray, or start fresh here. The running sum becomes the larger of the current element alone or the current element plus the previous running sum. Track the best running sum seen.

```java
static int maxSubArray(int[] nums) {
    int cur = nums[0], best = nums[0];
    for (int i = 1; i < nums.length; i++) {
        cur = Math.max(nums[i], cur + nums[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

Time	Space
O(n)	O(1)

WATCH OUT
Initialise both variables to the first element, not 0 — an all-negative array must return its largest (least negative) element, not 0.

Q6 Merge Intervals — combine all overlapping intervals.

</> CODE

SOLUTION

Given a list of intervals, merge every set of overlapping intervals and return the non-overlapping result.

```text
Input:  [[1,3], [2,6], [8,10], [15,18]]
Output: [[1,6], [8,10], [15,18]]
```

Sort by start time so overlaps are adjacent. Walk the sorted list, and for each interval either append it (no overlap with the last kept interval) or extend the last interval's end to cover it.

```java
static int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] iv : intervals) {
        int[] last = out.isEmpty() ? null : out.get(out.size() - 1);
        if (last == null || last[1] < iv[0])
            out.add(iv);
        else
            last[1] = Math.max(last[1], iv[1]);
    }
    return out.toArray(new int[0][]);
}
```

Time	Space
O(n log n)	O(n)

> **NOTE**
> The sort dominates the cost; the merge itself is a single linear pass.

Q7 3Sum — all unique triplets that sum to zero.

`</> CODE`

SOLUTION

Return all unique triplets in the array that add up to zero. The solution set must not contain duplicate triplets.

```text
Input:  nums = [-1, 0, 1, 2, -1, -4]
Output: [[-1, -1, 2], [-1, 0, 1]]
```

Sort the array, then fix each element and two-pointer the remainder for a pair summing to its negation. Sorting lets you both move the pointers by comparison and skip duplicates cleanly, avoiding repeated triplets.

```java
static List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;      // skip dup pivot
        int lo = i + 1, hi = nums.length - 1;
        while (lo < hi) {
            int sum = nums[i] + nums[lo] + nums[hi];
            if (sum == 0) {
                res.add(Arrays.asList(nums[i], nums[lo], nums[hi]));
                while (lo < hi && nums[lo] == nums[lo + 1]) lo++;
                while (lo < hi && nums[hi] == nums[hi - 1]) hi--;
                lo++; hi--;
            } else if (sum < 0) lo++;
            else hi--;
        }
    }
    return res;
}
```

Time	Space
O(n^2)	O(1)

> **NOTE**
> The space excludes the output list; sorting makes duplicate-skipping a simple neighbour comparison.

Q8 Product of Array Except Self — no division.

`</> CODE`

SOLUTION

Return an array where each position holds the product of all other elements, without using division and in linear time.

```text
Input:  [1, 2, 3, 4]
Output: [24, 12, 8, 6]
```

Each answer is the product of everything to its left times everything to its right. Compute prefix products in one forward pass into the output, then multiply by a running suffix product in a backward pass — using the output array itself for O(1) extra space.

java

```java
static int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] out = new int[n];
    out[0] = 1;
    for (int i = 1; i < n; i++)
        out[i] = out[i - 1] * nums[i - 1];   // prefix product
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        out[i] *= right;                     // multiply by suffix
        right *= nums[i];
    }
    return out;
}
```

Time	Space
O(n)	O(1)

TIP

Avoiding division also sidesteps the zero-element edge case that a divide-the-total approach must special-case.

SECTION 02
Strings

Q9 Valid Palindrome — ignore case and non-alphanumerics.

</> CODE

SOLUTION

Determine whether a string reads the same forward and backward, considering only letters and digits and ignoring case.

```text
Input:  "A man, a plan, a canal: Panama"
Output: true
```

Use two pointers from both ends. Skip any character that is not alphanumeric, then compare the two characters case-insensitively. If any pair differs it is not a palindrome.

```java
static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i))
                != Character.toLowerCase(s.charAt(j)))
            return false;
        i++; j--;
    }
    return true;
}
```

Time	Space
O(n)	O(1)

Q10 Valid Anagram — check if two strings are rearrangements.

</> CODE

SOLUTION

Return true if the second string is an anagram of the first — the same characters with the same counts, in any order.

```text
Input:  s = "anagram", t = "nagaram"
Output: true
```

Anagrams have identical character frequencies. For lowercase letters, a fixed 26-slot count array is enough: increment for one string and decrement for the other, then verify every count returned to zero.

```java
static boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count)
        if (c != 0) return false;
    return true;
}
```

Time	Space
O(n)	O(1)

TIP

For Unicode input beyond a-z, use a HashMap of character to count instead of a fixed array.

Q11 Longest Substring Without Repeating Characters — sliding window.

</> CODE

SOLUTION

Find the length of the longest substring that contains no repeated character.

```text
Input:  "abcabcbb"
Output: 3            (the substring "abc")
```

Maintain a window and remember the last index of each character. When you meet a character already inside the window, jump the window start to just past its previous occurrence. The best window length seen is the answer.

```java
static int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> last = new HashMap<>();
    int start = 0, best = 0;
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        if (last.containsKey(c) && last.get(c) >= start)
            start = last.get(c) + 1;
        last.put(c, i);
        best = Math.max(best, i - start + 1);
    }
    return best;
}
```

Time	Space
O(n)	O(k)

> **NOTE**
> The space O(k) is the size of the character set stored in the map; the window itself is tracked with two indices.

Q12 Group Anagrams — cluster words that are anagrams.

`</> CODE`

SOLUTION

Group a list of strings so that all anagrams of one another are in the same group.

```text
Input:  ["eat", "tea", "tan", "ate", "nat", "bat"]
Output: [["eat","tea","ate"], ["tan","nat"], ["bat"]]
```

Two words are anagrams exactly when their sorted characters match, so use the sorted string as a map key. Each word is appended to the list under its signature, and the map's values are the groups.

```java
static List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] c = s.toCharArray();
        Arrays.sort(c);
        String key = new String(c);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Time	Space
O(n k log k)	O(n k)

> **NOTE**
> Here n is the number of words and k the maximum word length; sorting each word costs k log k.

SECTION 03
Hashing

Q13 Contains Duplicate — detect any repeated value.

</> CODE

SOLUTION

Return true if any value appears at least twice in the array, and false if every element is distinct.

```text
Input:  [1, 2, 3, 1]
Output: true
```

Add each element to a hash set. The set's add method returns false when the value is already present, which immediately signals a duplicate without a second pass.

```java
static boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int n : nums)
        if (!seen.add(n)) return true;
    return false;
}
```

Time	Space
O(n)	O(n)

Q14 Subarray Sum Equals K — count subarrays with a given sum.

</> CODE

SOLUTION

Count how many contiguous subarrays sum exactly to k. Values may be negative.

```text
Input:  nums = [1, 1, 1], k = 2
Output: 2
```

Use running prefix sums. A subarray ending at the current index sums to k when some earlier prefix equals the current prefix minus k. Keep a map of how many times each prefix sum has occurred, seeding it with 0 seen once.

```java
static int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefix = new HashMap<>();
    prefix.put(0, 1);
    int sum = 0, count = 0;
    for (int n : nums) {
        sum += n;
        count += prefix.getOrDefault(sum - k, 0);
        prefix.merge(sum, 1, Integer::sum);
    }
    return count;
}
```

Time	Space
O(n)	O(n)

KEY IDEA

The seed prefix.put(0, 1) accounts for subarrays that start at index 0 and sum to k on their own.

Q15 Top K Frequent Elements — the k most common values.

</> CODE

SOLUTION

Return the k elements that appear most frequently in the array, in any order.

```text
Input:  nums = [1, 1, 1, 2, 2, 3], k = 2
Output: [1, 2]
```

Count frequencies with a map, then keep a size-k min-heap ordered by frequency. Offer every distinct value and drop the least frequent whenever the heap exceeds k, so the k most frequent remain.

java

```java
static int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);

    PriorityQueue<Integer> heap =
        new PriorityQueue<>((a, b) -> freq.get(a) - freq.get(b));
    for (int key : freq.keySet()) {
        heap.offer(key);
        if (heap.size() > k) heap.poll();
    }
    int[] out = new int[k];
    for (int i = k - 1; i >= 0; i--) out[i] = heap.poll();
    return out;
}
```

Time	Space
O(n log k)	O(n)

TIP

A bounded size-k heap beats sorting all counts (O(n log n)) when k is much smaller than n.

SECTION 04

Linked Lists

Q16 Reverse a Linked List — iterative pointer reversal.

</> CODE

SOLUTION

Reverse a singly linked list and return the new head. The node definition used throughout this section is a standard value-and-next node.

java

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}
```

Walk the list once, re-pointing each node's next to its predecessor while carrying a reference to the previous node. When you reach the end, the previous pointer is the new head.

java

```
static ListNode reverseList(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode next = cur.next;   // save the rest
        cur.next = prev;            // reverse the link
        prev = cur;                 // advance prev
        cur = next;                 // advance cur
    }
    return prev;
}
```

Time	Space
O(n)	O(1)

Q17 Linked List Cycle — detect a loop (Floyd's algorithm).

</> CODE

SOLUTION

Determine whether the linked list contains a cycle — a node whose next pointer eventually revisits an earlier node.

Use two pointers moving at different speeds: a slow pointer one step at a time and a fast pointer two steps. If there is a cycle the fast pointer laps the slow one and they meet; if the fast pointer reaches the end, there is no cycle. This uses no extra memory.

© 2026 ByteMart Digital

```java
static boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Time	Space
O(n)	O(1)

KEY IDEA

The two-pointer (tortoise and hare) trick detects a cycle in constant space — a hash set of visited nodes would also work but costs O(n) memory.

Q18 Merge Two Sorted Lists — combine into one sorted list.

</> CODE

SOLUTION

Merge two ascending sorted linked lists into a single ascending sorted list and return its head.

```text
Input:  1 -> 2 -> 4  and  1 -> 3 -> 4
Output: 1 -> 1 -> 2 -> 3 -> 4 -> 4
```

Use a dummy head to simplify edge cases. Repeatedly attach the smaller of the two current nodes to the tail, advancing that list. When one list runs out, attach the remainder of the other.

```java
static ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; }
        else                { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = (a != null) ? a : b;   // attach the leftover
    return dummy.next;
}
```

Time	Space
O(n + m)	O(1)

> **TIP**
> A dummy head node removes the special-case logic for choosing the very first element of the result.

Q19 Remove Nth Node From End — one-pass two-pointer.

</> CODE

SOLUTION

Remove the nth node counting from the end of the list and return the head. Aim to do it in a single pass.

```text
Input:  1 -> 2 -> 3 -> 4 -> 5,  n = 2
Output: 1 -> 2 -> 3 -> 5
```

Advance a fast pointer n steps ahead of a slow pointer, then move both until fast reaches the end. Slow now sits just before the target, so relink around it. A dummy head handles removal of the first node cleanly.

```java
static ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;   // skip the nth-from-end node
    return dummy.next;
}
```

Time	Space
O(n)	O(1)

Q20 Middle of the Linked List — fast and slow pointers.

</> CODE

SOLUTION

Return the middle node of the list. If there are two middle nodes (even length), return the second one.

```text
Input:  1 -> 2 -> 3 -> 4 -> 5
Output: node 3
```

Move a slow pointer one step and a fast pointer two steps at a time. When the fast pointer runs off the end, the slow pointer is exactly at the middle — a single pass with no length count needed.

```java
static ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```

Time	Space
O(n)	O(1)

SECTION 05

Stacks & Queues

Q21 Valid Parentheses — check bracket matching.

</> CODE

SOLUTION

Given a string of the brackets ( ) [ ] { }, decide whether every opening bracket is closed by the matching type in the correct order.

```text
Input:  "([{}])"     Output: true
Input:  "([)]"       Output: false
```

Push every opening bracket onto a stack. On a closing bracket, the stack top must be its matching opener — otherwise the string is invalid. At the end the stack must be empty, meaning nothing was left open.

```java
static boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    Map<Character, Character> pairs =
        Map.of(')', '(', ']', '[', '}', '{');
    for (char c : s.toCharArray()) {
        if (pairs.containsValue(c)) {
            stack.push(c);                      // opening bracket
        } else if (pairs.containsKey(c)) {      // closing bracket
            if (stack.isEmpty() || stack.pop() != pairs.get(c))
                return false;
        }
    }
    return stack.isEmpty();
}
```

Time	Space
O(n)	O(n)

TIP

Prefer ArrayDeque over the legacy Stack class — it is faster and not burdened by synchronization.

Q22 Min Stack — stack with constant-time minimum.

</> CODE

SOLUTION

Design a stack that supports push, pop, top, and retrieving the minimum element, all in constant time.

Store the running minimum alongside each value. When pushing, pair the value with the smaller of itself and the current minimum, so the minimum is always available at the top without scanning.

```java
class MinStack {
    private final Deque<int[]> stack = new ArrayDeque<>(); // {value, min}

    public void push(int x) {
        int min = stack.isEmpty() ? x : Math.min(x, stack.peek()[1]);
        stack.push(new int[]{ x, min });
    }
    public void pop()      { stack.pop(); }
    public int  top()      { return stack.peek()[0]; }
    public int  getMin()   { return stack.peek()[1]; }
}
```

Time	Space
O(1) ops	O(n)

KEY IDEA

Carrying the minimum in each frame keeps getMin at O(1); the alternative of scanning on demand would be O(n).

Q23 Daily Temperatures — monotonic stack.

</> CODE

SOLUTION

For each day, find how many days you must wait until a warmer temperature. If none, the answer for that day is 0.

```text
Input:  [73, 74, 75, 71, 69, 72, 76, 73]
Output: [1, 1, 4, 2, 1, 1, 0, 0]
```

Keep a stack of indices whose warmer day is still unknown, with temperatures decreasing down the stack. When a warmer day arrives, pop every colder index and record the distance. Each index is pushed and popped once, giving linear time.

```java
static int[] dailyTemperatures(int[] temps) {
    int[] answer = new int[temps.length];
    Deque<Integer> stack = new ArrayDeque<>();   // indices, decreasing temp
    for (int i = 0; i < temps.length; i++) {
        while (!stack.isEmpty() && temps[i] > temps[stack.peek()]) {
            int prev = stack.pop();
            answer[prev] = i - prev;
        }
        stack.push(i);
    }
    return answer;
}
```

Time	Space
O(n)	O(n)

SECTION 06

Recursion & Backtracking

Q24 Subsets — generate the power set.

</> CODE

SOLUTION

Return all possible subsets (the power set) of a set of distinct integers.

```text
Input:  [1, 2, 3]
Output: [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]
```

Backtracking builds subsets incrementally. At each position choose to include the current element or not, recording the running subset at every step and undoing the choice after exploring it (the classic add / recurse / remove pattern).

```java
static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

static void backtrack(int[] nums, int start, List<Integer> cur,
                      List<List<Integer>> result) {
    result.add(new ArrayList<>(cur));            // record current subset
    for (int i = start; i < nums.length; i++) {
        cur.add(nums[i]);                        // choose
        backtrack(nums, i + 1, cur, result);     // explore
        cur.remove(cur.size() - 1);              // un-choose
    }
}
```

Time	Space
O(n * 2^n)	O(n)

NOTE

There are 2^n subsets and copying each costs up to O(n); the O(n) space is the recursion depth plus the current path.

Q25 Permutations — all orderings of the input.

</> CODE

SOLUTION

Return every possible ordering (permutation) of a list of distinct integers.

```text
Input:  [1, 2, 3]
Output: [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]
```

Backtrack while tracking which elements are already used. When the current arrangement reaches full length, record it; otherwise try each unused element in turn, marking and unmarking it around the recursive call.

```java
static List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    helper(nums, new ArrayList<>(), new boolean[nums.length], result);
    return result;
}

static void helper(int[] nums, List<Integer> cur, boolean[] used,
                   List<List<Integer>> result) {
    if (cur.size() == nums.length) {
        result.add(new ArrayList<>(cur));
        return;
    }
    for (int i = 0; i < nums.length; i++) {
        if (used[i]) continue;
        used[i] = true;  cur.add(nums[i]);
        helper(nums, cur, used, result);
        used[i] = false; cur.remove(cur.size() - 1);
    }
}
```

Time	Space
O(n * n!)	O(n)

Q26 Generate Parentheses — all valid combinations.

</> CODE

SOLUTION

Given n pairs of parentheses, generate all well-formed (balanced) combinations.

```text
Input:  n = 3
Output: ((())), (()()), (())(), ()(()), ()()()
```

Build the string character by character, tracking how many open and close brackets are used. You may add an open bracket while some remain, and a close bracket only while it would not exceed the opens — which guarantees validity by construction.

```java
static List<String> generateParenthesis(int n) {
    List<String> result = new ArrayList<>();
    gen(new StringBuilder(), 0, 0, n, result);
    return result;
}

static void gen(StringBuilder sb, int open, int close, int n,
                List<String> result) {
    if (sb.length() == 2 * n) { result.add(sb.toString()); return; }
    if (open < n) {
        sb.append('(');
        gen(sb, open + 1, close, n, result);
        sb.deleteCharAt(sb.length() - 1);
    }
    if (close < open) {
        sb.append(')');
        gen(sb, open, close + 1, n, result);
        sb.deleteCharAt(sb.length() - 1);
    }
}
```

Time	Space
O(4^n / sqrt(n))	O(n)

KEY IDEA

Enforcing close < open at each step prunes invalid branches early, so only valid strings are ever built.

Q27 Combination Sum — numbers that add to a target (reuse allowed). </> CODE

SOLUTION

Given distinct positive candidates and a target, return all unique combinations that sum to the target. Each candidate may be used any number of times.

```text
Input:  candidates = [2, 3, 6, 7], target = 7
Output: [2, 2, 3], [7]
```

Backtrack, subtracting the chosen candidate from the remaining target. Passing the same start index (rather than the next) allows reuse; recursion stops when the remainder hits zero (a hit) or goes negative (a dead end).

java

```java
static List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    combo(candidates, 0, target, new ArrayList<>(), result);
    return result;
}

static void combo(int[] c, int start, int remain, List<Integer> cur,
                  List<List<Integer>> result) {
    if (remain == 0) { result.add(new ArrayList<>(cur)); return; }
    if (remain < 0)  return;
    for (int i = start; i < c.length; i++) {
        cur.add(c[i]);
        combo(c, i, remain - c[i], cur, result);   // i (not i+1) allows reuse
        cur.remove(cur.size() - 1);
    }
}
```

Time	Space
O(2^target)	O(target)

SECTION 07

Searching & Sorting

Q28 Binary Search — find a target in a sorted array.

</> CODE

SOLUTION

Return the index of a target in a sorted array, or -1 if it is absent.

```text
Input:  nums = [-1, 0, 3, 5, 9, 12], target = 9
Output: 4
```

Repeatedly halve the search range. Compare the middle element to the target and discard the half that cannot contain it. Compute the midpoint as low plus half the gap to avoid integer overflow on large bounds.

```java
static int binarySearch(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Time	Space
O(log n)	O(1)

WATCH OUT

Use lo + (hi - lo) / 2 rather than (lo + hi) / 2 — the naive sum can overflow int for very large indices.

Q29 Search in Rotated Sorted Array — modified binary search.

</> CODE

SOLUTION

A sorted array has been rotated at an unknown pivot. Find a target's index in O(log n), or return -1.

```text
Input:  nums = [4, 5, 6, 7, 0, 1, 2], target = 0
Output: 4
```

At each step, one half around the midpoint is still sorted. Determine which half is sorted, check whether the target lies within its ordered range, and recurse into the correct half — preserving the logarithmic time despite the rotation.

```java
static int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        if (nums[lo] <= nums[mid]) {                // left half sorted
            if (nums[lo] <= target && target < nums[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                                    // right half sorted
            if (nums[mid] < target && target <= nums[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

Time	Space
O(log n)	O(1)

Q30 Kth Largest Element — using a heap.

</> CODE

SOLUTION

Return the kth largest element in an unsorted array (the kth in sorted order, not the kth distinct).

```text
Input:  nums = [3, 2, 1, 5, 6, 4], k = 2
Output: 5
```

Maintain a min-heap of size k. Offer every element and, whenever the heap grows past k, remove the smallest. The heap then holds the k largest seen, and its root is the kth largest overall.

java

```java
static int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();  // min-heap
    for (int n : nums) {
        heap.offer(n);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Time	Space
O(n log k)	O(k)

TIP

A bounded size-k heap uses far less space than sorting the whole array when k is small.

SECTION 08
Trees

Q31 Maximum Depth of a Binary Tree — recursive height.

</> CODE

SOLUTION

Return the maximum depth: the number of nodes along the longest path from the root down to a leaf. The node type used in this section is a standard binary tree node.

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
```

The depth of a tree is one plus the greater of its two subtrees' depths, with an empty tree having depth zero. This maps directly to a short recursion.

```java
static int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Time	Space
O(n)	O(h)

NOTE

The space O(h) is the recursion stack depth, where h is the tree height — O(log n) if balanced, O(n) in the worst case.

Q32 Invert a Binary Tree — mirror it.

</> CODE

SOLUTION

Invert the tree so that the left and right children of every node are swapped, producing its mirror image.

```text
     4                    4
   /   \                /   \
  2     7     ->       7     2
 / \   / \            / \   / \
1   3 6   9          9   6 3   1
```

Recursively invert each subtree, then swap the two children at the current node. The base case of an empty node returns null.

```java
static TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode left  = invertTree(root.left);
    TreeNode right = invertTree(root.right);
    root.left  = right;
    root.right = left;
    return root;
}
```

Time	Space
O(n)	O(h)

Q33 Validate a Binary Search Tree — range checking.

</> CODE

SOLUTION

Determine whether a binary tree is a valid BST: every node's value must be greater than all values in its left subtree and less than all in its right subtree.

Checking only a node against its direct children is insufficient — a deep descendant can still violate the order. Instead pass down a valid (low, high) range that tightens as you descend; each node must fall strictly inside its range.

```java
static boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

static boolean validate(TreeNode node, Integer low, Integer high) {
    if (node == null) return true;
    if (low  != null && node.val <= low)  return false;
    if (high != null && node.val >= high) return false;
    return validate(node.left,  low, node.val)
        && validate(node.right, node.val, high);
}
```

Time	Space
O(n)	O(h)

WATCH OUT

The common mistake is comparing each node only to its immediate children — the range must be carried down the whole path.

Q34 Binary Tree Level Order Traversal — breadth-first search.

</> CODE

SOLUTION

Return the node values grouped by level, from top to bottom and left to right within each level.

```text
Tree:  3 / (9, 20) / 20 -> (15, 7)
Output: [[3], [9, 20], [15, 7]]
```

Use a queue for breadth-first traversal. At the start of each round, the queue holds exactly one level; process that many nodes, collecting their values and enqueuing their children for the next round.

java

```java
static List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left  != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Time	Space
O(n)	O(n)

KEY IDEA

Snapshotting the queue size at the start of each loop is what separates one level from the next.

Q35 Diameter of a Binary Tree — longest path between nodes.

</> CODE

SOLUTION

Return the length (in edges) of the longest path between any two nodes in the tree. The path may or may not pass through the root.

The longest path through any node is the sum of the heights of its left and right subtrees. Compute depths recursively, and while doing so update a running maximum of left-plus-right at every node. A one-element holder carries the best value out of the recursion.

java

```java
static int diameterOfBinaryTree(TreeNode root) {
    int[] best = new int[1];
    depth(root, best);
    return best[0];
}

static int depth(TreeNode node, int[] best) {
    if (node == null) return 0;
    int left  = depth(node.left, best);
    int right = depth(node.right, best);
    best[0] = Math.max(best[0], left + right);   // path through this node
    return 1 + Math.max(left, right);
}
```

Time	Space
O(n)	O(h)

KEY IDEA

Combining the diameter update with the depth computation keeps it a single O(n) traversal rather than recomputing heights per node.

SECTION 09

Bit Manipulation

Q36 Single Number — find the element that appears once.

</> CODE

SOLUTION

Every element appears exactly twice except one, which appears once. Find that single element in linear time and constant space.

```text
Input:  [4, 1, 2, 1, 2]
Output: 4
```

XOR is the key: a value XORed with itself is zero, and XOR is commutative. XORing all elements cancels every pair and leaves only the unique number — no extra memory required.

```java
static int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;   // pairs cancel to 0
    return result;
}
```

Time	Space
O(n)	O(1)

KEY IDEA

XOR gives a constant-space solution where a hash-set count would cost O(n) memory.

Q37 Number of 1 Bits — count set bits (Hamming weight).

</> CODE

SOLUTION

Count the number of 1 bits in the binary representation of an integer.

```text
Input:  11  (binary 1011)
Output: 3
```

The trick n AND (n - 1) clears the lowest set bit. Repeating it until the number becomes zero runs exactly once per set bit, so the loop count is the answer — faster than checking all 32 bit positions.

```java
static int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1);   // drop the lowest set bit
        count++;
    }
    return count;
}
```

Time	Space
O(bits set)	O(1)

SECTION 10

Dynamic Programming

Q38 Climbing Stairs — count distinct ways.

</> CODE

SOLUTION

You can climb 1 or 2 steps at a time. Count the distinct ways to reach the top of a staircase with n steps.

```text
Input:  n = 4
Output: 5
```

The ways to reach step n equal the ways to reach step n-1 plus those to reach step n-2 — the Fibonacci recurrence. Compute it bottom-up keeping only the last two values, avoiding exponential recursion.

```java
static int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

Time	Space
O(n)	O(1)

Q39 House Robber — max sum without adjacent picks.

</> CODE

SOLUTION

Given the amounts in a row of houses, find the maximum you can rob without taking from two adjacent houses.

```text
Input:  [2, 7, 9, 3, 1]
Output: 12            (rob houses 2 + 9 + 1)
```

At each house choose the better of skipping it (keep the previous best) or robbing it (its value plus the best from two houses back). Track the last two results, so only constant memory is needed.

```java
static int rob(int[] nums) {
    int prev = 0, curr = 0;
    for (int n : nums) {
        int take = prev + n;
        prev = curr;
        curr = Math.max(curr, take);
    }
    return curr;
}
```

Time	Space
O(n)	O(1)

Q40 Coin Change — fewest coins for an amount.

</> CODE

SOLUTION

Given coin denominations and a target amount, return the minimum number of coins needed, or -1 if the amount cannot be formed. You have an unlimited supply of each coin.

```text
Input:  coins = [1, 2, 5], amount = 11
Output: 3             (5 + 5 + 1)
```

Build up the best answer for every amount from 1 to the target. For each amount, try every coin that fits and take one plus the best solution for the remainder. Seed the table with an unreachable large value and 0 coins for amount 0.

```java
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);   // sentinel = unreachable
    dp[0] = 0;
    for (int i = 1; i <= amount; i++)
        for (int coin : coins)
            if (coin <= i)
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Time	Space
O(amount * coins)	O(amount)

NOTE
This is a bottom-up DP; a greedy 'largest coin first' strategy does not work for arbitrary denominations.

Q41 Longest Common Subsequence — 2D dynamic programming. </> CODE

SOLUTION

Find the length of the longest subsequence common to two strings. A subsequence keeps relative order but need not be contiguous.

```text
Input:  a = "abcde", b = "ace"
Output: 3             (the subsequence "ace")
```

Fill a table where each cell is the LCS length of the two prefixes. When the current characters match, extend the diagonal by one; otherwise carry the better of dropping one character from either string. The bottom-right cell holds the answer.

```java
static int longestCommonSubsequence(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (a.charAt(i - 1) == b.charAt(j - 1))
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[m][n];
}
```

Time	Space
O(m * n)	O(m * n)

TIP
The space can be reduced to O(n) by keeping only the previous row, since each cell depends only on the row above and the current row.

Q42 Word Break — can the string be segmented?

</> CODE

SOLUTION

Given a string and a dictionary of words, decide whether the string can be split into a sequence of one or more dictionary words.

```text
Input:  s = "applepenapple", dict = ["apple", "pen"]
Output: true
```

Let dp[i] mean the first i characters can be segmented. It is true when some earlier position j is segmentable and the piece from j to i is a dictionary word. Build the table forward; the final cell answers the question.

```java
static boolean wordBreak(String s, List<String> wordDict) {
    Set<String> words = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
    dp[0] = true;
    for (int i = 1; i <= s.length(); i++)
        for (int j = 0; j < i; j++)
            if (dp[j] && words.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
    return dp[s.length()];
}
```

Time	Space
O(n^2)	O(n)

NOTE

Substring extraction adds a factor of the word length; a trie can reduce repeated prefix checks for large dictionaries.